Numpy

Numpy is the core library for scientific computing in Python. It provides a high-performance multidimensional array object, and tools for working with these arrays.

Arrays

1. A numpy array is a grid of values, all of the same type, and is indexed by a tuple of nonnegative integers.
2. The number of dimensions is the rank of the array;
3. the shape of an array is a tuple of integers giving the size of the array along each dimension.
4. We can initialize numpy arrays from nested Python lists, and access elements using square brackets:

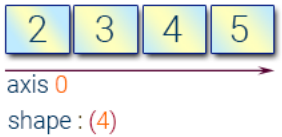
NumPy Basics

Operator	Description
np.array([1,2,3])	1d array
np.array([(1,2,3),(4,5,6)])	2d array
np.arange(start,stop,step)	range array

1D Array

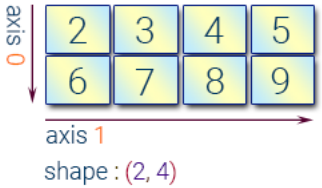
```
>>> import numpy as np
>>> x =  np.arange(2, 6).reshape(4)
>>> x
array( [ 2, 3, 4, 5 ] )

>>>
```



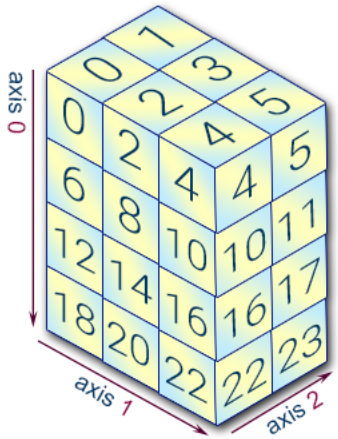
2D Array

```
>>> import numpy as np
>>> x =  np.arange(2, 10).reshape(2, 4)
>>> x
array( [ [ 2, 3, 4, 5 ] )
         [ 6, 7, 8, 9 ] ] )
>>>
```



3D Array

```
>>> import numpy as np
>>> x =  np.arange(24).reshape(4, 3, 2)
>>> x
array( [ [ [ 0, 1] ),      [ [ 6, 7] ),      [ [ 12, 13] ),    [ [ 18, 19] ),
           [ 2, 3] ),        [ 8, 9] ),        [ 14, 15] ),      [ 20, 21] ),
           [ 4, 5] ),        [ 10, 11] ),      [ 16, 17] ),      [ 22, 23] ] ] )
>>>
```



shape : (4, 3, 2)

© w3resource.com

Placeholders

Operator	Description
np.linspace(0,2,9)	Add evenly spaced values btw interval to array of length
np.zeros((1,2))	Create and array filled with zeros
np.ones((1,2))	Creates an array filled with ones
np.random.random((5,5))	Creates random array
np.empty((2,2))	Creates an empty array

Array

Syntax	Description
array.shape	Dimensions (Rows,Columns)
len(array)	Length of Array
array.ndim	Number of Array Dimensions
array.dtype	Data Type
array.astype(type)	Converts to Data Type
type(array)	Type of Array

Copying/Sorting

Operators	Description
np.copy(array)	Creates copy of array
other = array.copy()	Creates deep copy of array
array.sort()	Sorts an array
array.sort(axis=0)	Sorts axis of array

Array Manipulation

Adding or Removing Elements

Operator	Description
np.append(a,b)	Append items to array
np.insert(array, 1, 2, axis)	Insert items into array at axis 0 or 1
np.resize((2,4))	Resize array to shape(2,4)
np.delete(array,1,axis)	Deletes items from array

Combining Arrays

Operator	Description
np.concatenate((a,b),axis=0)	Concatenates 2 arrays, adds to end
np.vstack((a,b))	Stack array row-wise
np.hstack((a,b))	Stack array column wise

Splitting Arrays

Operator	Description
numpy.split()	Split an array into multiple sub-arrays.
np.array_split(array, 3)	Split an array in sub-arrays of (nearly) identical size
numpy.hsplit(array, 3)	Split the array horizontally at 3rd index

More

Operator	Description
other = ndarray.flatten()	Flattens a 2d array to 1d
array = np.transpose(other)	
array.T Transpose array	
inverse = np.linalg.inv(matrix)	Inverse of a given matrix
Mathematics	

Operations

Operator	Description
np.add(x,y)	
x + y	Addition
np.substract(x,y)	
x - y	Subtraction
np.divide(x,y)	
x / y	Division
np.multiply(x,y)	
x @ y	Multiplication
np.sqrt(x)	Square Root
np.sin(x)	Element-wise sine
np.cos(x)	Element-wise cosine
np.log(x)	Element-wise natural log
np.dot(x,y)	Dot product
np.roots([1,0,-4])	Roots of a given polynomial coefficients

Comparison

Operator	Description
==	Equal
!=	Not equal
<	Smaller than
>	Greater than
<=	Smaller than or equal
>=	Greater than or equal
np.array_equal(x,y)	Array-wise comparison

Basic Statistics

Operator	Description
np.mean(array)	Mean
np.median(array)	Median
array.corrcoef()	Correlation Coefficient
np.std(array)	Standard Deviation

More

Operator	Description
array.sum()	Array-wise sum
array.min()	Array-wise minimum value
array.max(axis=0)	Maximum value of specified axis
array.cumsum(axis=0)	Cumulative sum of specified axis

Slicing and Subsetting

Operator	Description
array[i]	1d array at index i
array[i,j]	2d array at index[i][j]
array[i<4]	Boolean Indexing, see Tricks
array[0:3]	Select items of index 0, 1 and 2
array[0:2,1]	Select items of rows 0 and 1 at column 1
array[:1]	Select items of row 0 (equals array[0:1, :])
array[1:2, :]	Select items of row 1
[comment]: <>	(array[1,...]
array[: :-1]	Reverses array

1. Installation

pip install numpy

2. Motivation

Numpy provides extension package to python for multi dimensional arrays.
Also known as array oriented computing.

creating array in numpy

```
import numpy as np

a = np.arange(10) #using arange function
print(a)

b = np.array([1,2,3,4,5]) #creating array from list
print(b)
```

why to use numpy ?

```
L = range(1000)
%timeit [i**2 for i in L]
```

```
A = np.arange(1000)
%timeit A**2
```

```
print("Numpy implementation is {0} times faster than lists implementation".format(272/1.25))
```

3. Creating arrays

3.1. Manual construction of the arrays

```
#create 1 - D array #vector
a = np.array([1,2,3,4,5])

a
```

```
#print dimensions
a.ndim
```

```
#print shape
a.shape
```

```
len(a)
```

```
#create 2-D array #matrix

b = np.array([[1,2,3],[4,5,6]]) #list of lists

b
```

```
b.ndim
```

```
b.shape
```

```
len(b) #size of first dimension or number of rows
```

```
#create 3 - D array #tensors
c = np.array([[[0,1],[2,3]],[[4,5],[6,7]]])
c
```

```
c.ndim
```

```
c.shape
```

```
len(c)
```

3.2. Functions for creating arrays

```
#using arange function
a = np.arange(10)
a
```

```
b = np.arange(1,10,2) #start, end, step
b
```

```
#using linspace #linearspace
c = np.linspace(5, 10, 5) #start, end, number of points
c
```

```
#common arrays
d = np.ones((3,3))
d
```

```
x = np.zeros((3,3))
x
```

```
y = np.eye(3) #creates a matrix with 1 as the diagonals and 0 as non-diagonal elements
y
```

```
z = np.eye(3,2)

z
```

```
a = np.diag([1,2,3,4]) #construct a diagonal array

print(a)
```

```
np.diag(a) #extracts the diagonal elements of matrix a
```

```
#creates a array using random
a = np.random.rand(4)

print(a)
```

4. Datatypes

```
#we can explicitly specify the required data type
a = np.arange(10, dtype='float')

print(a)
```

```
b = np.array([1+2j, 5+1j])

print(b.dtype)
```

```
c = np.array([True, False, True])

print(c.dtype)
```

5. Indexing and slicing

5.1. Indexing

```
a = np.arange(10)

#print(a)
print(a[5])
print(a[-1])
```

```
b = np.diag([1,2,3])

#print(b)
print(b[2,2])
```

```
b[2,1] = 10 #asigning value to a index

print(b)
```

5.2. Slicing

```
a = np.arange(10)

print(a[1:10:2]) #[start_value: end_value(exclusive): step]
```

```
b = np.arange(10)

b[5:] = 10 #assign 10 from index 5 to end

print(b)
```

```
c = np.arange(5)

b[5:] = c[::-1] #assign in reverse order

print(b)
```

6. Copies and views

Slicing operation creates a view on the original array which is just a way of accessing the data. Thus, the original array is not copied in the memory. You can use **np.may_share_memory()** to check whether two arrays share the same memory block.

```
a = np.arange(10)

b = a[::2]

np.shares_memory(a,b)
```

```
b[0] = 10

print(b)
print(a) #a is also updated, since is shares the same location in memory
```

```
c = a[::2].copy() #force the copy

np.shares_memory(a,c)
```

```
c[0] = 5

print(c)
print(a)
```

7. Fancy Indexing

7.1 Using Boolean mask

```
a = np.random.randint(0,20,15)

print(a)


mask = (a % 2 == 0)


even_numbers = a[mask]

print(even_numbers)


a[mask]=-1 #it can be very useful to assign a new value to sub array

print(a)
```

7.2. Using Integer Array

```
a = np.arange(0,100,10)

print(a)


b = a[[2,3,5,2,4]]

print(b)


a[[9,7]] = -200

print(a)
print(b)
```

8. Numerical Operations on numpy

8.1. Element wise operations

```
a = np.arange(10)

print(a+1)
```

```
print(a ** 2)


b = np.ones(10) + 1
print("b = ", b)

print("a - b = ",a-b)


print(a*b)


#Matrix Multiplication

c = np.diag([1,2,3,4])

print(c)
print("*"*100)
print(c*c)
print("*"*100)
print(c.dot(c))


# element comparisions

a = np.array([1,2,5,4])
b = np.array([6,2,9,4])

print(a==b)


print(a>b)


print(a<b)


print(a<=b)


#array wise comparision

print(np.array_equal(a,b))


c = np.array([1,2,5,4])

print(np.array_equal(a,c))
```

Logical Operations

```
a = np.array([1,0,0,1],dtype='bool')
b = np.array([0,1,0,1],dtype='bool')

print(np.logical_or(a,b))
```

```
print(np.logical_and(a,b))
```

```
print(np.logical_not(a))
```

Transcendental Functions:

```
a = np.arange(5)+1
```

```
print(np.sin(a))
```

```
print(np.log(a))
```

```
print(np.exp(a))
```

Shape Mismatch:

```
a = np.array([1,2,3,4])
```

```
b = np.array([5, 10])
```

```
print(a+b)
```

8.2. Basic Reductions

```
x = np.array([1,2,3,4])
```

```
print(np.sum(x))
```

```
y = np.array([[1,2],[3,4]])
```

```
print(y)
print("*"*100)
print(y.T)
```

```
print(y.sum(axis=0)) #column wise sum
```

```
print(y.sum(axis=1)) #row wise sum
```

other reductions

```
print(y.min())
```

```
print(y.max())
```

```
print(y.argmin()) #index of minimum element
```

```
print(y.argmax()) #index of maximum element
```

Logical reductions

```
print(np.all([True, False, False])) #logical and
```

```
print(np.any([True, False, False])) #logical or
```

```
a = np.zeros((50,50))

print(np.any(a!=0)) #checks whether any element in a is not equal to zero
```

Statistics

```
x = np.arange(1,10)

print(np.mean(x))
```

```
print(np.median(x))
```

```
y = np.array([[1,2,3],[4,5,6]])

print(np.mean(y,axis=0)) #column wise mean
print(np.mean(y,axis=1)) #row wise mean
```

```
print(np.std(x))
```

Example:

```
data = np.loadtxt("dataset/populations.txt")
```

```
print(data)
```

```
data.shape
```

```
year, hare, lynx, carrot = data.T
```

```
print(year)
```

```
populations = data[:,1:]
```

```
print(populations)
```

```
populations.std(axis=0)
```

```
populations.mean(axis=0)
```

```
#which species has the highest population each year

print(np.argmax(populations, axis=1))
```

9. Broadcasting

```
from IPython.display import Image

Image("photos/broadcasting.PNG")
```

```
a = np.tile(np.arange(0, 40, 10),(3,1)) #replicate rows 3 times and columns 1 time # change columns
print(a)
print("*"*100)

a = a.T
print(a)
```

```
b = np.array([0,1,2])

print(b)
```

```
c = a + b

print(c)
```

```
a = np.arange(0,40,10)
print(a.shape)
```

```
a = a[:, np.newaxis] #adds a new axis
print(a.shape)
```

```
print(a)
```

```
print(a+b)
```

10. Array Manipulation

10.1. Flattening

```
a = np.array([[1,2,3],[4,5,6]])
print(a)
```

```
print(a.ravel())
```

```
print(a.T)
```

```
print(a.T.ravel())
```

10.2. Reshaping

```
print(a.shape)
```

```
b = a.ravel()
print(b.shape)
```

```
b = b.reshape(2,3)
print(b.shape)
```

10.3. Adding a new dimension

```
z = np.arange(1,10,2)
print(z.shape)
```

```
z = z[:,np.newaxis]
print(z.shape)
```

```
z = z[:,np.newaxis]
print(z.shape)
```

10.4. Dimension Shuffling

```
a = np.arange(4*3*2)
print(a)
```

```
a = a.reshape(4,3,2) # 4 matrices of 3 rows and 2 columns each
print(a)
```

```
a[3,1,1]
```

10.5. Resizing

```
a = np.arange(5)
a.resize((10,))
```

10.6. Sorting

```
a = np.array([[5,4,3],[2,1,9]])
print(a)

b = np.sort(a, axis=1)
print(b)

c = np.sort(a, axis=0)
print(c)

d = a.ravel()
print(d)

print(np.sort(d))
```

11. Exercises

(Type you code in the below cell)

1. **Write a NumPy program to convert a list of numeric value into a one-dimensional NumPy array.**

Expected Output:

Original List: [12.23, 13.32, 100, 36.32]

One-dimensional NumPy array: [12.23 13.32 100. 36.32]

2. **Write a NumPy program to create a 3x3 matrix with values ranging from 2 to 10.**

Expected Output:

[[2 3 4] [5 6 7] [8 9 10]]

3. *Write a NumPy program to sort an along the first, last axis of an array.**

Sample array: [[2,5],[4,4]]

Expected Output:

Original array: [[4 6] [2 1]]

Sort along the first axis: [[2 1] [4 6]]

Sort along the last axis: [[1 2] [4 6]]

4. **Write a NumPy program to create a contiguous flattened array.**

Expected Output:

Original array: [[10 20 30] [20 40 50]]

New flattened array: [10 20 30 20 40 50]

5. **Write a NumPy program to display all the dates for the month of March, 2017.**

Expected Output:

March, 2017

['2017-03-01' '2017-03-02' '2017-03-03' '2017-03-04' '2017-03-05' '2017-03-06' '2017-03-07' '2017-03-08' '2017-03-09' '2017-03-10' '2017-03-11' '2017-03-12' '2017-03-13' '2017-03-14' '2017-03-15' '2017-03-16' '2017-03-17' '2017-03-18' '2017-03-19' '2017-03-20' '2017-03-21' '2017-03-22' '2017-03-23' '2017-03-24' '2017-03-25' '2017-03-26' '2017-03-27' '2017-03-28' '2017-03-29' '2017-03-30' '2017-03-31']

6. **Write a NumPy program to generate six random integers between 10 and 30.**

Expected Output:

[20 28 27 17 28 29]

7. **Write a NumPy program to create a 5x5 array with random values and find the minimum and maximum values.**

Expected Output

Original Array:

[[0.96336355 0.12339131 0.20295196 0.37243578 0.88105252] [0.93228246 0.67470158 0.38103235 0.32242645 0.40610231] [0.3113495 0.31688 0.79189089 0.08676434 0.60829874] [0.30360149 0.94316317 0.98142491 0.77222542 0.51532195] [0.97392305 0.16669609 0.81377917 0.2165645 0.00121611]]

Minimum and Maximum Values:

0.00121610921757 0.981424910368

```
Start coding or generate with AI.
```

8. **Write a NumPy program to get the powers of an array values element-wise.**

Note: First array elements raised to powers from second array

Expected Output:

Original array

[0 1 2 3 4 5 6]

First array elements raised to powers from second array, element-wise:

[0 1 8 27 64 125 216]

```
Start coding or generate with AI.
```

9. **Write a NumPy program to create a structured array from given student name, height, class and their data types. Now sort the array on height.**

Expected Output:

Original array:

[(b'James', 5, 48.5) (b'Nail', 6, 52.5) (b'Paul', 5, 42.1) (b'Pit', 5, 40.11)]

Sort by height

[(b'Pit', 5, 40.11) (b'Paul', 5, 42.1) (b'James', 5, 48.5) (b'Nail', 6, 52.5)]